

Lab 4 Report

Designing and Managing Data Lakes

Step 0: Setup

0a: Create the Data Lake Directory Structure

Setting up the physical directory hierarchy for our three-zone data lake (Bronze/Silver/Gold) on the local filesystem to simulate object storage like S3 or HDFS.

```
# Define base path
$baseDir = "D:\EFREI\Data_Engineering\LAB4\dataLake"

# Create directories
New-Item -ItemType Directory -Force -Path "$baseDir\raw\source=kafka\topic=sensor-events"
New-Item -ItemType Directory -Force -Path "$baseDir\curated\domain=iot"
New-Item -ItemType Directory -Force -Path "$baseDir\consumption\use_case=sensor_averages"

# Verify structure
Get-ChildItem -Path $baseDir -Recurse -Directory
```

The document uses bash brace expansion, which isn't native to PowerShell. We use `New-Item -Force` to recursively create the necessary nested directories.

```
(base) PS C:\Users\xback20040219> conda activate bigdata
(bigdata) PS C:\Users\xback20040219> cd "D:\EFREI\Data_Engineering\LAB4"
(bigdata) PS D:\EFREI\Data_Engineering\LAB4> $baseDir = "D:\EFREI\Data_Engineering\LAB4\datalake"
(bigdata) PS D:\EFREI\Data_Engineering\LAB4> New-Item -ItemType Directory -Force -Path "$baseDir\raw\source=kafka\topic=sensor-events"

    目录: D:\EFREI\Data_Engineering\LAB4\datalake\raw\source=kafka

Mode                LastWriteTime         Length Name
----                -
d-----          2026/5/14      10:45             topic=sensor-events

(bigdata) PS D:\EFREI\Data_Engineering\LAB4> New-Item -ItemType Directory -Force -Path "$baseDir\curated\domain=iot"

    目录: D:\EFREI\Data_Engineering\LAB4\datalake\curated

Mode                LastWriteTime         Length Name
----                -
d-----          2026/5/14      10:45             domain=iot

(bigdata) PS D:\EFREI\Data_Engineering\LAB4> New-Item -ItemType Directory -Force -Path "$baseDir\consumption\use_case=sensor_averages"

    目录: D:\EFREI\Data_Engineering\LAB4\datalake\consumption

Mode                LastWriteTime         Length Name
----                -
d-----          2026/5/14      10:45             use_case=sensor_averages

(bigdata) PS D:\EFREI\Data_Engineering\LAB4> Get-ChildItem -Path $baseDir -Recurse -Directory

    目录: D:\EFREI\Data_Engineering\LAB4\datalake

Mode                LastWriteTime         Length Name
----                -
d-----          2026/5/14      10:45             consumption
d-----          2026/5/14      10:45             curated
d-----          2026/5/14      10:45             raw
```

0b: Ensure the Kafka Cluster and Topic Are Ready

Verifying that the Kafka source is running and contains sensor event messages to stream into our data lake.

```
# Verify containers are running
docker compose ps

# List topics
docker exec kafka1 kafka-topics --bootstrap-server kafka1:29092 --list

# Produce sample messages (assuming producer.py exists from a previous lab)
python producer.py
```

Ensuring `sensor-events` exists and has data is critical so Spark has messages to read.

```
(bigdata) PS D:\EFREI\Data_Engineering\LAB4> docker compose ps
time="2026-05-14T10:53:46+08:00" level=warning msg="D:\\EFREI\\Data_Engineering\\LAB4\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
NAME        IMAGE                                  COMMAND                                SERVICE    CREATED        STATUS        PORTS
kafka-ui    provectuslabs/kafka-ui:latest        /bin/sh -c 'java -D...'              kafka-ui   5 minutes ago  Up 5 minutes  0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
kafka1      confluentinc/cp-kafka:7.5.0          /etc/confluent/docker/run            kafka1     5 minutes ago  Up 5 minutes  0.0.0.0:9092->9092/tcp, [::]:9092->9092/tcp
kafka2      confluentinc/cp-kafka:7.5.0          /etc/confluent/docker/run            kafka2     5 minutes ago  Up 5 minutes  0.0.0.0:9094->9094/tcp, [::]:9094->9094/tcp
kafka3      confluentinc/cp-kafka:7.5.0          /etc/confluent/docker/run            kafka3     5 minutes ago  Up 5 minutes  0.0.0.0:9096->9096/tcp, [::]:9096->9096/tcp
(bigdata) PS D:\EFREI\Data_Engineering\LAB4> docker exec kafka1 kafka-topics --bootstrap-server kafka1:29092 --delete --topic sensor-events
(bigdata) PS D:\EFREI\Data_Engineering\LAB4> docker exec kafka1 kafka-topics --bootstrap-server kafka1:29092 --create --topic sensor-events --partitions 3 --replication-factor 3
Created topic sensor-events.
(bigdata) PS D:\EFREI\Data_Engineering\LAB4> docker exec kafka1 kafka-topics --bootstrap-server kafka1:29092 --list
sensor-events
```

```
(bigdata) PS D:\EFREI\Data_Engineering\LAB4> python producer.py
```

```
Sent to partition 2, offset 28  
Sent to partition 0, offset 22  
Sent to partition 2, offset 29  
Sent to partition 2, offset 30  
Sent to partition 2, offset 31  
Sent to partition 2, offset 32  
Sent to partition 2, offset 33  
Sent to partition 2, offset 34  
Sent to partition 0, offset 23  
Sent to partition 2, offset 35  
Sent to partition 0, offset 24  
Sent to partition 2, offset 36  
Sent to partition 2, offset 37  
Sent to partition 0, offset 25  
Sent to partition 0, offset 26  
Sent to partition 2, offset 38  
Sent to partition 2, offset 39  
Sent to partition 0, offset 27  
Sent to partition 2, offset 40  
Sent to partition 2, offset 41  
Sent to partition 2, offset 42  
Sent to partition 2, offset 43  
Sent to partition 2, offset 44  
Sent to partition 2, offset 45  
Sent to partition 2, offset 46  
Sent to partition 2, offset 47  
Sent to partition 2, offset 48  
Sent to partition 2, offset 49  
Sent to partition 2, offset 50  
Sent to partition 2, offset 51  
Sent to partition 2, offset 52  
Sent to partition 0, offset 28  
Sent to partition 0, offset 29  
Sent to partition 2, offset 53  
Sent to partition 2, offset 54  
Sent to partition 2, offset 55  
Sent to partition 2, offset 56  
Sent to partition 0, offset 30  
Sent to partition 2, offset 57  
Sent to partition 2, offset 58  
Sent to partition 2, offset 59  
Sent to partition 2, offset 60  
Sent to partition 2, offset 61  
Sent to partition 2, offset 62  
Sent to partition 0, offset 31  
Sent to partition 0, offset 32  
Sent to partition 2, offset 63  
Sent to partition 2, offset 64  
Sent to partition 0, offset 33
```

```
Sent to partition 2, offset 65
Producer closed.
(bigdata) PS D:\EFREI\Data_Engineering\LAB4>
```

Step 1: Raw Zone - Land Data from Kafka

(Create a Python file named `datalake_pipeline.py` and add the following blocks)

1a: Imports and SparkSession

Initializing the Spark application and defining core configuration paths.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    col, from_json, to_timestamp, expr, year, month, dayofmonth, hour, lit
)
from pyspark.sql.types import (
    StructType, StructField, StringType, DoubleType, LongType, BooleanType
)
import os

# Set up local paths relative to the current working directory
LAKE_ROOT = "./datalake"
RAW_PATH = f"{LAKE_ROOT}/raw/source=kafka/topic=sensor-events"
CURATED_PATH = f"{LAKE_ROOT}/curated/domain=iot"
CONSUME_PATH = f"{LAKE_ROOT}/consumption/use_case=sensor_averages"
CKPT_RAW = "./datalake-ckpt/raw"
CKPT_CUR = "./datalake-ckpt/curated"

KAFKA_BROKERS = "localhost:9092,localhost:9094,localhost:9096"
TOPIC = "sensor-events"

spark = SparkSession.builder \
    .appName("Data Lake Pipeline") \
    .master("local[*]") \
    .config("spark.jars.packages", "org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.3") \
    .config("spark.sql.shuffle.partitions", "3") \
    .getOrCreate()

spark.sparkContext.setLogLevel("WARN")
```

```

datalake_pipeline.py > ...
1  # 1a: imports and SparkSession
2
3  from pyspark.sql import SparkSession
4  from pyspark.sql.functions import (
5      col, from_json, to_timestamp, expr, year, month, dayofmonth, hour, lit
6  )
7  from pyspark.sql.types import (
8      StructType, StructField, StringType, DoubleType, LongType, BooleanType
9  )
10 import os
11
12 # Set up local paths relative to the current working directory
13 LAKE_ROOT = "./datalake"
14 RAW_PATH = f"{LAKE_ROOT}/raw/source=kafka/topic=sensor-events"
15 CURATED_PATH = f"{LAKE_ROOT}/curated/domain=iot"
16 CONSUME_PATH = f"{LAKE_ROOT}/consumption/use_case=sensor_averages"
17 CKPT_RAW = "./datalake-ckpt/raw"
18 CKPT_CUR = "./datalake-ckpt/curated"
19
20 KAFKA_BROKERS = "localhost:9092,localhost:9094,localhost:9096"
21 TOPIC = "sensor-events"
22
23 spark = SparkSession.builder \
24     .appName("Data Lake Pipeline") \
25     .master("local[*]") \
26     .config("spark.jars.packages", "org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.3") \
27     .config("spark.sql.shuffle.partitions", "3") \
28     .getOrCreate()
29
30 spark.sparkContext.setLogLevel("WARN")

```

1b: Define Schema and Read Raw Kafka Stream

Defining the expected structure of incoming data and connecting Spark to the Kafka topic as a streaming source.

```

SENSOR_SCHEMA = StructType([
    StructField("sensor", StringType(), False),
    StructField("value", DoubleType(), False),
    StructField("unit", StringType(), True),
    StructField("timestamp", LongType(), False),
    StructField("source", StringType(), True),
    StructField("anomaly", BooleanType(), True),
])

raw_kafka = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", KAFKA_BROKERS) \
    .option("subscribe", TOPIC) \
    .option("startingOffsets", "earliest") \
    .option("failOnDataLoss", "false") \
    .load()

```

```

32 # 1b: Define Schema and Read Raw Kafka Stream
33 SENSOR_SCHEMA = StructType([
34     StructField("sensor", StringType(), False),
35     StructField("value", DoubleType(), False),
36     StructField("unit", StringType(), True),
37     StructField("timestamp", LongType(), False),
38     StructField("source", StringType(), True),
39     StructField("anomaly", BooleanType(), True),
40 ])
41
42 raw_kafka = spark.readStream \
43     .format("kafka") \
44     .option("kafka.bootstrap.servers", KAFKA_BROKERS) \
45     .option("subscribe", TOPIC) \
46     .option("startingOffsets", "earliest") \
47     .option("failOnDataLoss", "false") \
48     .load()

```

1c: Write Raw Zone (JSON format, partitioned by date)

Saving the raw bytes from Kafka exactly as they arrived (as JSON), partitioned by the ingestion timestamp. This acts as our immutable audit trail.

```

raw_for_lake = raw_kafka.select(
    col("value").cast("string").alias("raw_json"),
    col("partition").alias("kafka_partition"),
    col("offset").alias("kafka_offset"),
    col("timestamp").alias("ingestion_ts"),
    year(col("timestamp")).alias("year"),
    month(col("timestamp")).alias("month"),
    dayofmonth(col("timestamp")).alias("day"),
    hour(col("timestamp")).alias("hour")
)

query_raw = raw_for_lake.writeStream \
    .outputMode("append") \
    .format("json") \
    .option("path", RAW_PATH) \
    .option("checkpointLocation", CKPT_RAW) \
    .partitionBy("year", "month", "day", "hour") \
    .trigger(processingTime="30 seconds") \
    .start()

```

```

50 # 1c: Write Raw Zone (JSON format, partitioned by date)
51 raw_for_lake = raw_kafka.select(
52     col("value").cast("string").alias("raw_json"),
53     col("partition").alias("kafka_partition"),
54     col("offset").alias("kafka_offset"),
55     col("timestamp").alias("ingestion_ts"),
56     year(col("timestamp")).alias("year"),
57     month(col("timestamp")).alias("month"),
58     dayofmonth(col("timestamp")).alias("day"),
59     hour(col("timestamp")).alias("hour")
60 )
61
62 query_raw = raw_for_lake.writeStream \
63     .outputMode("append") \
64     .format("json") \
65     .option("path", RAW_PATH) \
66     .option("checkpointLocation", CKPT_RAW) \
67     .partitionBy("year", "month", "day", "hour") \
68     .trigger(processingTime="30 seconds") \
69     .start()

```

Step 2: Curated Zone - Clean, Parse, and Partition

2a: Parse and Validate

Extracting the JSON string into typed columns and applying data quality gates (e.g., filtering out nulls and physically impossible sensor values).

```

parsed = raw_kafka.select(
    from_json(col("value").cast("string"), SENSOR_SCHEMA).alias("d"),
    col("partition"),
    col("offset")
).select(
    col("d.sensor").alias("sensor_type"),
    col("d.value"),
    col("d.unit"),
    col("d.anomaly").alias("is_anomaly"),
    to_timestamp(expr("d.timestamp / 1000")).alias("event_time"),
    col("partition"),
    col("offset")
).filter(col("sensor_type").isNotNull()) \
    .filter(col("value").isNotNull()) \
    .filter(col("value").between(-100, 2000))

```

`between(-100, 2000)` enforces data quality. Event time (when the sensor actually recorded the data) is extracted here.

```
71 # 2a: Parse and Validate
72 parsed = raw_kafka.select(
73     from_json(col("value").cast("string"), SENSOR_SCHEMA).alias("d"),
74     col("partition"),
75     col("offset")
76 ).select(
77     col("d.sensor").alias("sensor_type"),
78     col("d.value"),
79     col("d.unit"),
80     col("d.anomaly").alias("is_anomaly"),
81     to_timestamp(expr("d.timestamp / 1000")).alias("event_time"),
82     col("partition"),
83     col("offset")
84 ).filter(col("sensor_type").isNotNull()) \
85     .filter(col("value").isNotNull()) \
86     .filter(col("value").between(-100, 2000))
```

2b: Add Partitioning Columns from Event Time

Creating explicit date columns from the `event_time` to be used for Hive-style directory partitioning.

```
curated = parsed \
    .withColumn("year", year(col("event_time"))) \
    .withColumn("month", month(col("event_time"))) \
    .withColumn("day", dayofmonth(col("event_time")))
```

```
88 # 2b: Add Partitioning Columns from Event Time
89 curated = parsed \
90     .withColumn("year", year(col("event_time"))) \
91     .withColumn("month", month(col("event_time"))) \
92     .withColumn("day", dayofmonth(col("event_time")))
```

2c & 2d: Write Curated Zone (Parquet) and Start

Writing the cleaned data in a columnar format (Parquet) compressed with Snappy, and telling Spark to wait for the streams to finish.

```
query_curated = curated.writeStream \
    .outputMode("append") \
    .format("parquet") \
    .option("path", CURATED_PATH) \
    .option("checkpointLocation", CKPT_CUR) \
    .option("compression", "snappy") \
    .partitionBy("sensor_type", "year", "month", "day") \
    .trigger(processingTime="30 seconds") \
    .start()
```

```

print("Data lake pipeline running. Ctrl+C to stop.")
try:
    spark.streams.awaitAnyTermination()
except KeyboardInterrupt:
    print("Stopping all queries...")
    for q in spark.streams.active:
        q.stop()

```

```

94  # 2c & 2d: Write Curated Zone (Parquet) and Start
95  query_curated = curated.writeStream \
96      .outputMode("append") \
97      .format("parquet") \
98      .option("path", CURATED_PATH) \
99      .option("checkpointLocation", CKPT_CUR) \
100     .option("compression", "snappy") \
101     .partitionBy("sensor_type", "year", "month", "day") \
102     .trigger(processingTime="30 seconds") \
103     .start()
104
105  print("Data lake pipeline running. Ctrl+C to stop.")
106  try:
107      spark.streams.awaitAnyTermination()
108  except KeyboardInterrupt:
109      print("Stopping all queries...")
110      for q in spark.streams.active:
111          q.stop()

```

2e: Run `datalake_pipeline.py`

```
python datalake_pipeline.py
```

```
(bigdata) PS D:\EFRET\Data_Engineering\LAB4> python datalake_pipeline.py
:: loading settings :: url = jar:file:/D:/APP/anaconda/envs/bigdata/Lib/site-packages/pyspark/jars/ivy-2.5.1.jar!/org/apache/ivy/core/settings/ivysettings.xml
Ivy Default Cache set to: C:\Users\xbac20040219\ivy2\cache
The jars for the packages stored in: C:\Users\xbac20040219\ivy2\jars
org.apache.spark#spark-sql-kafka-0-10_2.12 added as a dependency
:: resolving dependencies :: org.apache.spark#spark-submit-parent-3a746c3b-6b55-4132-8e7b-e9acc75cc752;1.0
  confs: [default]
    found org.apache.spark#spark-sql-kafka-0-10_2.12:3.5.3 in central
    found org.apache.spark#spark-token-provider-kafka-0-10_2.12:3.5.3 in central
    found org.apache.kafka#kafka-clients:3.4.1 in central
    found org.lz4#lz4-java:1.8.0 in central
    found org.xerial.snappy#snappy-java:1.1.10.5 in central
    found org.slf4j#slf4j-api:2.0.7 in central
    found org.apache.hadoop#hadoop-client-runtime:3.3.4 in central
    found org.apache.hadoop#hadoop-client-api:3.3.4 in central
    found commons-logging#commons-logging:1.1.3 in central
    found com.google.code.findbugs#jsr305:3.0.0 in local-m2-cache
    found org.apache.commons#commons-pool2:2.11.1 in central
  :: resolution report :: resolve 1170ms :: artifacts dl 246ms
    :: modules in use:
    com.google.code.findbugs#jsr305:3.0.0 from local-m2-cache in [default]
    commons-logging#commons-logging:1.1.3 from central in [default]
    org.apache.commons#commons-pool2:2.11.1 from central in [default]
    org.apache.hadoop#hadoop-client-api:3.3.4 from central in [default]
    org.apache.hadoop#hadoop-client-runtime:3.3.4 from central in [default]
    org.apache.kafka#kafka-clients:3.4.1 from central in [default]
    org.apache.spark#spark-sql-kafka-0-10_2.12:3.5.3 from central in [default]
    org.apache.spark#spark-token-provider-kafka-0-10_2.12:3.5.3 from central in [default]
    org.lz4#lz4-java:1.8.0 from central in [default]
    org.slf4j#slf4j-api:2.0.7 from central in [default]
    org.xerial.snappy#snappy-java:1.1.10.5 from central in [default]
  -----
  | conf | number | modules | search | evicted | artifacts | |
|---|---|---|---|---|---|---|
  | default | 11 | 0 | 0 | 0 | 11 | 0 |
  :: retrieving :: org.apache.spark#spark-submit-parent-3a746c3b-6b55-4132-8e7b-e9acc75cc752
    confs: [default]
    0 artifacts copied, 11 already retrieved (0KB/53ms)
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/05/14 11:11:45 WARN ResolveWriteToStream: spark.sql.adaptive.enabled is not supported in streaming DataFrames/Datasets and will be disabled.
26/05/14 11:11:46 WARN ResolveWriteToStream: spark.sql.adaptive.enabled is not supported in streaming DataFrames/Datasets and will be disabled.
26/05/14 11:11:46 WARN AdminClientConfig: These configurations '[key.serializer, value.serializer, enable.auto.commit, max.poll.records, auto.offset.reset]' were supplied but are not used yet.
Data lake pipeline running. Ctrl+C to stop.
26/05/14 11:11:47 WARN AdminClientConfig: These configurations '[key.serializer, value.serializer, enable.auto.commit, max.poll.records, auto.offset.reset]' were supplied but are not used yet.
```

Wait about 1–2 minutes to ensure that the previously generated messages have been written to disk.
Press Ctrl+C to stop the script.

Step 3: Consumption Zone - Build Aggregated Dataset

Run this in a new PySpark interactive shell after stopping Step 2.

Enter the standard Python interactive environment

```
python
```

Manually initialize Spark with Kafka dependencies

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("Interactiveshell") \
    .master("local[*]") \
    .config("spark.jars.packages", "org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.3") \
    .getOrCreate()

spark.sparkContext.setLogLevel("WARN")
```

```
>>> spark = (SparkSession.builder
...   .appName("InteractiveShell")
...   .master("local[*]")
...   .config("spark.jars.packages", "org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.3")
...   .getOrCreate())
:: loading settings :: url = jar:file:/D:/APP/anaconda/envs/bigdata/Lib/site-packages/pyspark/jars/ivy-2.5.1.jar!/org/apache/ivy/core/settings/ivysettings.xml
Ivy Default Cache set to: C:\Users\xback20040219\ivy2\cache
The jars for the packages stored in: C:\Users\xback20040219\ivy2\jars
org.apache.spark#spark-sql-kafka-0-10_2.12 added as a dependency
:: resolving dependencies :: org.apache.spark#spark-submit-parent-d70da7bd-420f-4de1-b67f-4c4c8ec120f4;1.0
  confs: [default]
    found org.apache.spark#spark-sql-kafka-0-10_2.12:3.5.3 in central
    found org.apache.spark#spark-token-provider-kafka-0-10_2.12:3.5.3 in central
    found org.apache.kafka#kafka-clients:3.4.1 in central
    found org.lz4#lz4-java:1.8.0 in central
    found org.xerial.snappy#snappy-java:1.1.10.5 in central
    found org.slf4j#slf4j-api:2.0.7 in central
    found org.apache.hadoop#hadoop-client-runtime:3.3.4 in central
    found org.apache.hadoop#hadoop-client-api:3.3.4 in central
    found commons-logging#commons-logging:1.1.3 in central
    found com.google.code.findbugs#jsr305:3.0.0 in local-m2-cache
    found org.apache.commons#commons-pool2:2.11.1 in central
:: resolution report :: resolve 630ms :: artifacts dl 225ms
:: modules in use:
  com.google.code.findbugs#jsr305:3.0.0 from local-m2-cache in [default]
  commons-logging#commons-logging:1.1.3 from central in [default]
  org.apache.commons#commons-pool2:2.11.1 from central in [default]
  org.apache.hadoop#hadoop-client-api:3.3.4 from central in [default]
  org.apache.hadoop#hadoop-client-runtime:3.3.4 from central in [default]
  org.apache.kafka#kafka-clients:3.4.1 from central in [default]
  org.apache.spark#spark-sql-kafka-0-10_2.12:3.5.3 from central in [default]
  org.apache.spark#spark-token-provider-kafka-0-10_2.12:3.5.3 from central in [default]
  org.lz4#lz4-java:1.8.0 from central in [default]
  org.slf4j#slf4j-api:2.0.7 from central in [default]
  org.xerial.snappy#snappy-java:1.1.10.5 from central in [default]

```

conf	modules				artifacts	
	number	search	downloaded	evicted	number	downloaded
default	11	0	0	0	11	0

```

:: retrieving :: org.apache.spark#spark-submit-parent-d70da7bd-420f-4de1-b67f-4c4c8ec120f4
  confs: [default]
  0 artifacts copied, 11 already retrieved (0kB/43ms)
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
>>>
>>> spark.sparkContext.setLogLevel("WARN")
>>>

```

Creating the Gold layer by running a batch job over the curated data to build a pre-aggregated, business-ready table.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import avg, min, max, count, sum, col

spark = SparkSession.builder.appName("ConsumptionJob").master("local[*]").getOrCreate()

LAKE_ROOT = "./datalake"
CURATED_PATH = f"{LAKE_ROOT}/curated/domain=iot"
CONSUME_PATH = f"{LAKE_ROOT}/consumption/use_case=sensor_averages"

# Read curated
curated_df = spark.read.format("parquet").load(CURATED_PATH)
print(f"Curated zone: {curated_df.count()} records")

# Aggregate
daily_agg = curated_df \
    .groupBy("sensor_type", "year", "month", "day") \
    .agg(
        count("value").alias("record_count"),
        avg("value").alias("avg_value"),
        min("value").alias("min_value"),
        max("value").alias("max_value"),
        sum(col("is_anomaly").cast("int")).alias("anomaly_count")
    ).orderBy("sensor_type", "year", "month", "day")

# Write consumption
daily_agg.write \
    .mode("overwrite") \
```

```
.partitionBy("sensor_type", "year", "month") \
.parquet(CONSUME_PATH)
```

```
print("Consumption zone written.")
daily_agg.show(20, truncate=False)
```

```
>>> from pyspark.sql import SparkSession
>>> from pyspark.sql.functions import avg, min, max, count, sum, col
>>>
>>> spark = SparkSession.builder.appName("ConsumptionJob").master("local[*]").getOrCreate()
26/05/14 11:31:10 WARN SparkSession: Using an existing Spark session; only runtime SQL configurations will take effect.
>>>
>>> LAKE_ROOT = "./datalake"
>>> CURATED_PATH = f"{LAKE_ROOT}/curated/domain=iot"
>>> CONSUME_PATH = f"{LAKE_ROOT}/consumption/use_case=sensor_averages"
>>>
>>> # Read curated
>>> curated_df = spark.read.format("parquet").load(CURATED_PATH)
>>> print(f"Curated zone: {curated_df.count()} records")
Curated zone: 100 records
>>>
>>> # Aggregate
>>> daily_agg = curated_df \
...   .groupBy("sensor_type", "year", "month", "day") \
...   .agg(
...     count("value").alias("record_count"),
...     avg("value").alias("avg_value"),
...     min("value").alias("min_value"),
...     max("value").alias("max_value"),
...     sum(col("is_anomaly").cast("int")).alias("anomaly_count")
...   ).orderBy("sensor_type", "year", "month", "day")
>>>
>>> # Write consumption
>>> daily_agg.write \
...   .mode("overwrite") \
...   .partitionBy("sensor_type", "year", "month") \
...   .parquet(CONSUME_PATH)
>>>
>>> print("Consumption zone written.")
Consumption zone written.
>>> daily_agg.show(20, truncate=False)
```

sensor_type	year	month	day	record_count	avg_value	min_value	max_value	anomaly_count
humidity	2026	5	14	34	24.24470588235294	11.98	39.54	NULL
pressure	2026	5	14	34	25.221470588235295	10.46	39.92	NULL
temperature	2026	5	14	32	27.8609375	12.27	39.69	NULL

Step 4: Query with Spark SQL and Partition Pruning

4a: Register the Curated Dataset as a Table

Exposing the Parquet files as a virtual SQL table for querying.

```
curated_df = spark.read.parquet(CURATED_PATH)
curated_df.createOrReplaceTempView("sensor_curated")
curated_df.printSchema()
```

```

>>> curated_df = spark.read.parquet(CURATED_PATH)
>>> curated_df.createOrReplaceTempView("sensor_curated")
>>> curated_df.printSchema()
root
|-- value: double (nullable = true)
|-- unit: string (nullable = true)
|-- is_anomaly: boolean (nullable = true)
|-- event_time: timestamp (nullable = true)
|-- partition: integer (nullable = true)
|-- offset: long (nullable = true)
|-- sensor_type: string (nullable = true)
|-- year: integer (nullable = true)
|-- month: integer (nullable = true)
|-- day: integer (nullable = true)

```

4b: Query 1: Basic Partition Pruning

Executing business queries. Query 1 demonstrates filtering on partition columns (`sensor_type`, `year`, `month`).

```

# Query 1
result1 = spark.sql("""
    SELECT sensor_type, day,
           ROUND(AVG(value), 2) AS avg_value,
           ROUND(MIN(value), 2) AS min_value,
           ROUND(MAX(value), 2) AS max_value,
           COUNT(*) AS total_records
    FROM sensor_curated
    WHERE sensor_type = 'temperature'
           AND year = 2026 AND month = 5
    GROUP BY sensor_type, day
    ORDER BY day
""")
result1.show()

```

```
>>> result1 = spark.sql("""
...     SELECT sensor_type, day,
...           ROUND(AVG(value), 2) AS avg_value,
...           ROUND(MIN(value), 2) AS min_value,
...           ROUND(MAX(value), 2) AS max_value,
...           COUNT(*) AS total_records
...     FROM sensor_curated
...     WHERE sensor_type = 'temperature'
...           AND year = 2026 AND month = 5
...     GROUP BY sensor_type, day
...     ORDER BY day
... """)
>>> result1.show()
```

sensor_type	day	avg_value	min_value	max_value	total_records
temperature	14	27.86	12.27	39.69	32

4c: Query 2: Anomaly Analysis

Query 2 calculates anomaly percentages.

```
# Query 2
result2 = spark.sql("""
    SELECT sensor_type,
           COUNT(*) AS total,
           SUM(CAST(is_anomaly AS INT)) AS anomalies,
           ROUND(100.0 * SUM(CAST(is_anomaly AS INT)) / COUNT(*), 2) AS anomaly_pct
    FROM sensor_curated
    GROUP BY sensor_type
    ORDER BY anomaly_pct DESC
""")
result2.show()
```

```
>>> result2 = spark.sql("""
...     SELECT sensor_type,
...            COUNT(*) AS total,
...            SUM(CAST(is_anomaly AS INT)) AS anomalies,
...            ROUND(100.0 * SUM(CAST(is_anomaly AS INT)) / COUNT(*), 2) AS anomaly_pct
...     FROM sensor_curated
...     GROUP BY sensor_type
...     ORDER BY anomaly_pct DESC
... """)
>>> result2.show()
```

sensor_type	total	anomalies	anomaly_pct
humidity	34	NULL	NULL
pressure	34	NULL	NULL
temperature	32	NULL	NULL

```
>>>
```

4d: Measure the Pruning Benefit

Comparing the execution time of a full table scan versus a pruned scan.

```
import time

start = time.time()
spark.sql("SELECT COUNT(*) FROM sensor_curated").collect()
full_scan_time = time.time() - start

start = time.time()
spark.sql("SELECT COUNT(*) FROM sensor_curated WHERE sensor_type = 'temperature'").collect()
pruned_time = time.time() - start

print(f"Full scan time: {full_scan_time:.2f}s")
print(f"Pruned scan time: {pruned_time:.2f}s")
print(f"Speedup: {full_scan_time/pruned_time:.1f}x")
```

```
>>> import time
>>>
>>> start = time.time()
>>> spark.sql("SELECT COUNT(*) FROM sensor_curated").collect()
[Row(count(1)=100)]
>>> full_scan_time = time.time() - start
>>>
>>> start = time.time()
>>> spark.sql("SELECT COUNT(*) FROM sensor_curated WHERE sensor_type = 'temperature'").collect()
[Row(count(1)=32)]
>>> pruned_time = time.time() - start
>>>
>>> print(f"Full scan time: {full_scan_time:.2f}s")
Full scan time: 0.49s
>>> print(f"Pruned scan time: {pruned_time:.2f}s")
Pruned scan time: 0.11s
>>> print(f"Speedup: {full_scan_time/pruned_time:.1f}x")
Speedup: 4.3x
>>>
```

Reflection Questions

1. You have a dataset with columns: user_id (10M distinct values), country (50 values), event_date, event_type (8 values). Which columns would you choose for partitioning and why?

I would choose `event_date` and `event_type`. `event_date` is excellent because most analytical queries filter by time, aligning with data retention policies. `event_type` has low cardinality (8 values) and is likely a common filter.

2. Explain why the raw zone uses ingestion time for its directory partitioning, while the curated zone uses event time. In what situation could a gap between the two times indicate a problem?

The raw zone uses ingestion time because it answers the operational question "when did this data arrive in our system?", which is crucial for debugging pipeline delays. The curated zone uses event time because business queries need to know when the real-world measurement actually happened. A large gap between ingestion time and event time indicates a pipeline delay, network latency, or an outage where the source system held onto messages before finally successfully sending them to Kafka.

3. What is the "small file problem" and what are its two main consequences on query performance?

The small file problem occurs when a data lake is populated with thousands of tiny files (often caused by frequent streaming micro-batches). Its main consequences are:

High metadata overhead: Systems like HDFS/S3 struggle to process the metadata of 10,000 x 1MB files compared to 100 x 100MB files, slowing down reads significantly.

Reduced write/read performance and limited parallelism due to excessive task scheduling overhead in Spark.

4. A new field `firmware_version` is added to the sensor payload starting January 20. Old Parquet files (before Jan 20) do not have this field. What happens when Spark reads both old and new files together?

Thanks to Parquet's schema evolution support, Spark will seamlessly read both files together. For the old files (before Jan 20), Spark will automatically return `null` values for the `firmware_version` column, without failing or requiring the old files to be rewritten.

5. You delete the curated zone checkpoint and restart the streaming pipeline. What happens to the raw zone checkpoint? Does data get duplicated in the curated zone?

The raw zone checkpoint remains unaffected, as it is managed by a separate streaming query with its own checkpoint directory (`CKPT_RAW`). However, because the curated zone checkpoint was deleted, Spark loses track of which offsets it had previously processed from `raw_kafka`. When restarted, the curated streaming query will process from the beginning ("earliest"), resulting in data being duplicated in the curated Parquet files.